

Questões Respondidas - Programação Web Avançada

Gabarito comentado

Programacao Web Avancada
Universidade de Vila Velha

*Este documento foi gerado a partir do banco de questoes
estruturado no site de estudos (index.html).
Cada questao traz o topico e a unidade correspondente.*

Sumário

- Unidade 1 - Python essencial (20 questões)
- Unidade 2 - HTTP, REST e http.server (21 questões)
- Unidade 3 - Flask e primeira aplicação (11 questões)
- Unidade 4 - Application Factory e Contexto (15 questões)

Este é o gabarito. A resposta correta está marcada com (X) e há uma explicação curta abaixo. Use após tentar responder por conta própria.

Unidade 1 - Python essencial

1.

[Sintaxe básica]

Qual a diferença entre `=` e `==` em Python?

- () A) Não há diferença, são sinônimos
- (X) B) `=` atribui valor; `==` compara igualdade
- () C) `=` compara; `==` atribui
- () D) `==` só funciona com strings
- () E) `==` é usado apenas em funções

Por que: `=` é atribuição ("cola essa etiqueta no valor"). `==` é comparação ("essa etiqueta está nesse valor?"). Usar `=` num `if` gera `SyntaxError`.

2.

[Estruturas de dados]

Qual estrutura remove duplicatas automaticamente e não preserva ordem?

- () A) list
- () B) tuple
- (X) C) set
- () D) dict
- () E) frozenset apenas

Por que: `set` = coleção não ordenada, mutável, sem duplicatas. Ideal para testes de pertinência e operações matemáticas de conjuntos (união, interseção...).

3.

[POO]

O que é um decorator em Python?

- () A) Um comentário especial
- (X) B) Uma função que recebe outra função e devolve uma "envelopada" com comportamento antes/depois
- () C) Um tipo de variável
- () D) Uma classe herdada
- () E) Uma sintaxe para importações

Por que: Decorator = função que recebe uma função e retorna outra (o wrapper). `@dec` é açúcar sintático

para ``func = dec(func)``. Muito usado no Flask.

4.

[POO]

Qual mecanismo é ativado ao prefixar um atributo com dois underscores (ex: ``__preco``)?

- ☐ A) Encriptação automática
- ☒ B) **Name Mangling (renomeia para ``_NomeClasse__atributo``)**
- ☐ C) Proteção absoluta contra acesso
- ☐ D) Compilação estática
- ☐ E) Cache em memória

Por que: É Name Mangling. Python renomeia para ``_NomeClasse__atributo``. Não é proteção real - é convenção de "não mexa daqui de fora". Ainda dá para acessar como ``p._Produto__preco``.

5.

[POO]

Para que serve o decorator ``@property``?

- ☐ A) Criar métodos de classe
- ☒ B) **Permitir que um método seja acessado como se fosse atributo, com controle sobre leitura/escrita**
- ☐ C) Herdar de outra classe
- ☐ D) Definir tipo estático
- ☐ E) Cachear o resultado

Por que: ``@property`` disfarça um método de atributo. Com ``@x.setter`` você intercepta escritas para validar. Ex: ``p.preco = -5`` pode disparar `ValueError`.

6.

[POO]

Qual argumento por convenção o ``@classmethod`` recebe?

- ☐ A) `self`
- ☒ B) **`cls`**
- ☐ C) `this`
- ☐ D) `class`
- ☐ E) `me`

Por que: ``self`` é para métodos de instância. ``cls`` é a referência à própria classe, usada em ``@classmethod``.

7.

[POO]

``@staticmethod``:

- ☐ A) Recebe `self`
- ☐ B) Recebe `cls`
- ☒ C) **Não recebe nem `self` nem `cls` - é essencialmente uma função dentro da classe**
- ☐ D) É executado apenas uma vez
- ☐ E) É proibido em Python

Por que: ``@staticmethod`` não recebe nem ``self`` nem ``cls``. É uma função comum "morando" na classe por

organização, sem acesso a estado.

8.

[POO]

O que é `super()` em Python?

- ☐ A) Cria uma nova classe
- ☒ B) **Invoca métodos da classe pai a partir da filha**
- ☐ C) Torna a classe mais rápida
- ☐ D) Elimina herança
- ☐ E) É um decorator

Por que: `super()` permite chamar métodos da classe pai. Típico em `super().__init__(...)` para inicializar a "parte do pai" antes da parte específica da filha.

9.

[POO]

O que é Duck Typing?

- ☐ A) Um tipo de dado específico do Python
- ☒ B) **Compatibilidade entre objetos determinada pelo comportamento (métodos disponíveis), não pela hierarquia formal**
- ☐ C) Um mecanismo de compilação de tipos
- ☐ D) Uma restrição de tipos em runtime
- ☐ E) Um framework para tipagem estrita

Por que: "Se parece pato, anda como pato e faz quack, é pato". Python aceita o objeto se ele tiver os métodos necessários - não exige que seja formalmente de uma classe específica.

10.

[Estruturas de dados]

Qual dessas afirmações sobre `frozenset` está CORRETA?

- ☐ A) É um dicionário congelado
- ☒ B) **É um conjunto imutável, e pode ser usado como chave de dict**
- ☐ C) Preserva ordem de inserção
- ☐ D) Aceita duplicatas
- ☐ E) Não existe em Python

Por que: `frozenset` é imutável. Como imutável, é hashable, então pode ser chave de dicionário. Um `set` normal não pode.

11.

[POO]

Numa `@dataclass(frozen=True)`, tentar alterar um atributo após criação:

- ☐ A) Funciona normalmente
- ☒ B) **Lança `FrozenInstanceError`**
- ☐ C) Cria uma nova instância
- ☐ D) Ignora silenciosamente
- ☐ E) Requer recompilação

Por que: ``frozen=True`` torna a instância imutável. Qualquer tentativa de ``p.x = 99`` dispara ``FrozenInstanceError``.

12.

[Operadores]

Qual o resultado de ``10 + 3 * 4 ** 2``?

☐ A) 208

☒ B) 58

☐ C) 148

☐ D) 82

☐ E) 100

Por que: Ordem: ``**`` primeiro -> ``4**2 = 16``. Depois ``*`` -> ``3*16 = 48``. Por fim ``+`` -> ``10+48 = 58``.

13.

[Funções]

Uma função sem ``return`` explícito retorna:

☐ A) 0

☐ B) False

☒ C) None

☐ D) Lança exceção

☐ E) A última expressão avaliada

Por que: Sem ``return``, a função retorna implicitamente ``None`` (equivalente a null em outras linguagens).

14.

[Operadores]

Qual o resultado de ``-101 // 7`` em Python?

☐ A) -14

☒ B) -15

☐ C) -14.4

☐ D) Erro

☐ E) 15

Por que: Divisão inteira em Python arredonda para baixo, não trunca. ``-101/7 = -14.428...`` -> arredondando para baixo -> ``-15``.

15.

[Type hints]

O que são type hints em Python?

☐ A) Restrições de tipo que causam erro em runtime se violadas

☒ B) Anotações opcionais úteis para legibilidade e ferramentas de análise estática

☐ C) Um tipo especial de decorator

☐ D) Um mecanismo de compilação

☐ E) Variáveis globais tipadas

Por que: Type hints (PEP 484) são anotações opcionais. Não mudam nada em runtime. Existem para: legibilidade, autocomplete e análise estática (mypy/pyright).

16.

[Estrutura de arquivo]

O que o bloco `if __name__ == "__main__":` faz?

- () A) Ativa modo debug do Python
- (X) B) **Executa o código apenas quando o arquivo é rodado diretamente, não quando importado**
- () C) Cria uma função `main()` automaticamente
- () D) Faz o Python ignorar o arquivo
- () E) É obrigatório em todo arquivo Python

Por que: A variável `__name__` vira `"__main__"` quando o arquivo é executado direto (`python arquivo.py`), mas vira o nome do módulo quando importado. Permite que o arquivo funcione como `lib` e como `script`.

17.

[Funções]

Em `def saudacao(nome, horario="dia"):`, o que é `"dia"`?

- () A) O tipo esperado do parâmetro
- (X) B) **O valor padrão do parâmetro se não for passado**
- () C) Uma docstring
- () D) Um decorator implícito
- () E) Uma variável global

Por que: É o valor padrão. Se você chama `saudacao("Ana")`, `horario` recebe `"dia"` automaticamente.

18.

[Funções]

O que `**kwargs` recebe em uma função?

- () A) Argumentos posicionais como tupla
- (X) B) **Argumentos nomeados (keyword) como dicionário**
- () C) Um único argumento obrigatório
- () D) Apenas strings
- () E) Argumentos padrão

Por que: `*args` junta posicionais numa tupla. `**kwargs` junta os nomeados num dict.

19.

[Estruturas de dados]

O que caracteriza uma tupla em Python?

- () A) É mutável e ordenada
- (X) B) **É sequência imutável, definida pela vírgula (parênteses são opcionais na maioria dos contextos)**
- () C) É não ordenada
- () D) É sinônimo de lista
- () E) Só aceita tipos primitivos

Por que: Tupla = sequência imutável. O que a define é a vírgula, não os parênteses. `x = 5,` já é tupla. Para tupla de 1 elemento, a vírgula é obrigatória: `(5,)`.

20.

[Estruturas de dados]

Em Python 3.7+, dicionários preservam a ordem de:

- ☐ A) Alfabetica das chaves
- ☒ B) **Inserção**
- ☐ C) Numérica dos valores
- ☐ D) Não preservam ordem
- ☐ E) Aleatória

Por que: Desde Python 3.7 (PEP 468 + implementação do CPython) dicionários preservam a ordem de inserção. Antes era necessário `collections.OrderedDict`.

Unidade 2 - HTTP, REST e http.server

1.

[Códigos HTTP]

O que significa o código HTTP 200?

- ☐ A) Erro do cliente
- ☐ B) Redirecionamento
- ☒ C) **Requisição processada com sucesso (OK)**
- ☐ D) Erro do servidor
- ☐ E) Recurso não encontrado

Por que: `200 OK`: requisição processada com sucesso. No caso de GET, o corpo da resposta contém o recurso.

2.

[REST]

Qual dessas NÃO é uma restrição do REST?

- ☐ A) Cliente-Servidor
- ☐ B) Ausência de estado (stateless)
- ☐ C) Cache
- ☐ D) Interface uniforme
- ☒ E) **Autenticação obrigatória**

Por que: Autenticação não é restrição REST. As 6 são: Cliente-Servidor, Stateless, Cache, Interface Uniforme, Sistema em Camadas, Código sob Demanda.

3.

[REST]

Qual restrição REST é EXPLICITAMENTE opcional?

- ☐ A) Cliente-Servidor
- ☐ B) Stateless
- ☐ C) Cache
- ☐ D) Sistema em camadas

(X) E) Código sob demanda

Por que: Código sob demanda é a única opcional entre as 6. As outras cinco são obrigatórias.

4.

[REST]

Em REST, um "recurso" é:

- ☐ A) Um registro obrigatório de banco de dados
- ☒ B) Uma abstração de informação identificável no sistema**
- ☐ C) Sempre uma classe Python
- ☐ D) Um arquivo HTML
- ☐ E) Uma função remota

Por que: Recurso é conceito identificado por URI. Pode ser um usuário, produto, coleção, imagem - abstrato. Não se confunde com registro de banco, que é detalhe de implementação.

5.

[REST]

Qual a diferença entre recurso e representação?

- ☐ A) Não há diferença
- ☒ B) Recurso é o conceito identificado por URI; representação é a forma de transferir (JSON, HTML, XML)**
- ☐ C) Recurso é HTML e representação é JSON
- ☐ D) São sinônimos
- ☐ E) Recurso é remoto, representação é local

Por que: Livro (recurso) pode ter várias representações: impresso, ebook, audiobook. Em REST: `/api/users/42` é o recurso; o JSON retornado é a representação.

6.

[http.server]

No módulo `http.server`, qual método é chamado automaticamente para uma requisição GET?

- ☐ A) `handle_get`
- ☐ B) `request_get`
- ☒ C) `do_GET`**
- ☐ D) `on_get`
- ☐ E) `process_get`

Por que: Por convenção, o método HTTP se traduz em `do\_XXX()`. GET -> `do\_GET`. POST -> `do\_POST`. Etc.

7.

[http.server]

Qual a sequência CORRETA ao construir uma resposta HTTP no `BaseHTTPRequestHandler`?

- () A) wfile.write -> send_header -> send_response
- (X) B) **send_response -> send_header -> end_headers -> wfile.write**
- () C) end_headers -> send_response -> wfile.write
- () D) send_header -> wfile.write -> send_response
- () E) Só wfile.write basta

Por que: Ordem sagrada: `send\_response(status)` -> `send\_header(...)` (uma ou várias) -> `end\_headers()` -> `wfile.write(body)`. Fora de ordem = resposta HTTP quebrada.

8.

[http.server]

Qual classe do `http.server` oferece concorrência via threads?

- () A) AsyncHTTPServer
- (X) B) **ThreadingHTTPServer**
- () C) MultiHTTPServer
- () D) ConcurrentServer
- () E) ForkedServer

Por que: `ThreadingHTTPServer` trata cada requisição em uma thread. Ainda não é servidor de produção - falta segurança, escala, robustez.

9.

[Códigos HTTP]

O que diferencia 400 Bad Request de 404 Not Found?

- () A) Nenhuma diferença
- (X) B) **400 = a requisição em si é inválida (formato errado); 404 = a requisição é válida mas o recurso não existe**
- () C) 400 = servidor caiu; 404 = permissão negada
- () D) Ambos indicam sucesso
- () E) São ambos para redirecionamento

Por que: `/users/abc` -> 400 (id malformatado). `/users/999` onde 999 não existe -> 404. Erro clássico de confundir os dois.

10.

[REST]

Uma API é RESTful quando:

- () A) Só faz CRUD
- (X) B) **Segue coerentemente as restrições do estilo REST**
- () C) Usa HTTPS
- () D) Retorna JSON
- () E) É escrita em Python

Por que: RESTful != CRUD. Uma API RESTful segue as 6 restrições. CRUD (create/read/update/delete) descreve apenas operações em dados.

11.

[HTTP]

Qual cabeçalho HTTP informa ao cliente o tipo de conteúdo do corpo?

- ☐ A) Content-Length
- ☒ B) Content-Type
- ☐ C) Content-Encoding
- ☐ D) Response-Type
- ☐ E) Body-Type

Por que: `Content-Type` descreve o media type do corpo, ex.: `application/json`, `text/html; charset=utf-8`.

12.

[Códigos HTTP]

Código 404 significa:

- ☐ A) Erro interno do servidor
- ☐ B) Sucesso
- ☐ C) Requisição inválida
- ☒ D) Recurso não encontrado no servidor
- ☐ E) Não autorizado

Por que: `404 Not Found`: o servidor não encontrou o recurso solicitado. Não confunda com 400 (requisição mal formada).

13.

[WSGI]

WSGI é:

- ☐ A) Um servidor web específico
- ☒ B) Uma especificação de interface entre servidores web e aplicações Python (PEP 3333)
- ☐ C) Um framework Python
- ☐ D) Um protocolo HTTP alternativo
- ☐ E) Uma linguagem

Por que: WSGI (PEP 3333) é a "tomada padrão" que permite servidores como Gunicorn falarem com apps como Flask/Django. Não é servidor nem framework - é a interface.

14.

[http.server]

O módulo `http.server` é adequado para:

- ☐ A) Produção em alta escala
- ☒ B) Estudo, protótipos e testes locais
- ☐ C) Substituir Flask em produção
- ☐ D) APIs de missão crítica
- ☐ E) Sistemas com alta segurança

Por que: É didático/experimental. A própria documentação alerta: não use em produção. Faltam segurança, escalabilidade, features avançadas.

15.

[Códigos HTTP]

A classe 4xx representa:

- ☐ A) Sucesso
- ☐ B) Redirecionamento
- ☒ C) **Erros associados à requisição do cliente**
- ☐ D) Erros do servidor
- ☐ E) Respostas informativas

Por que: 4xx = erros do lado cliente. 5xx = erros do lado servidor. 3xx = redirecionamento. 2xx = sucesso. 1xx = informativa.

16.

[Códigos HTTP]

Qual código indica que a requisição foi processada e resultou em criação de novo recurso?

- ☐ A) 200 OK
- ☒ B) **201 Created**
- ☐ C) 204 No Content
- ☐ D) 400 Bad Request
- ☐ E) 301 Moved

Por que: `201 Created` tem sentido mais específico que 200: além do sucesso, indica que um recurso foi criado. Costuma vir com header `Location`.

17.

[Códigos HTTP]

Código 204 significa:

- ☐ A) OK com conteúdo
- ☒ B) **No Content - sucesso sem corpo de resposta**
- ☐ C) Redirect
- ☐ D) Bad Request
- ☐ E) Sem sentido

Por que: `204 No Content`: sucesso, mas sem corpo. Típico de DELETE bem-sucedido. Não envie corpo JSON com 204.

18.

[Métodos HTTP]

Qual a intenção semântica de `GET`?

- ☐ A) Criar recurso
- ☒ B) **Obter representação de um recurso**
- ☐ C) Deletar recurso
- ☐ D) Substituir recurso
- ☐ E) Enviar formulário

Por que: GET = obter representação (leitura). Não deve ter efeitos colaterais no servidor.

19.

[Métodos HTTP]

O método `PUT`:

- ☐ A) Só obtém dados
- ☒ B) **Solicita criação ou substituição da representação do recurso-alvo**
- ☐ C) Só cria novo com id auto-gerado
- ☐ D) Deleta o recurso
- ☐ E) Redireciona

Por que: PUT substitui (ou cria, se não existir) a representação do recurso identificado pela URI. Diferente do POST, que costuma criar recurso subordinado com id gerado pelo servidor.

20.

[Métodos HTTP]

`DELETE` tem qual intenção?

- ☐ A) Criar recurso
- ☐ B) Obter recurso
- ☐ C) Substituir
- ☒ D) **Solicitar remoção do recurso**
- ☐ E) Autenticar

Por que: DELETE remove o recurso. Retorno típico: `204 No Content`.

21.

[REST]

Quantas restrições arquiteturais compõem o REST?

- ☐ A) 3
- ☐ B) 4
- ☐ C) 5
- ☒ D) **6**
- ☐ E) 7

Por que: REST tem 6 restrições: Cliente-Servidor, Stateless, Cache, Interface Uniforme, Sistema em Camadas, Código sob Demanda (opcional).

Unidade 3 - Flask e primeira aplicação

1.

[venv]

Qual das alternativas melhor descreve a função de um ambiente virtual em Python?

- ☐ A) Permitir executar o Flask sem o terminal
- ☐ B) Aumentar o desempenho do interpretador
- ☐ C) Executar múltiplas aplicações Flask simultaneamente
- ☐ D) Proteger o código contra falhas de segurança
- ☒ E) **Isolar dependências e evitar conflitos entre projetos**

Por que: venv cria uma instalação isolada do Python por projeto: cada um com suas versões de pacotes, sem conflitos, sem precisar de sudo.

2.

[Proxies]

O que faz `current_app.config.get("CONFIG_NAME", "desconhecido")`?

- ☐ A) Cria um novo ambiente virtual
- ☒ B) **Recupera o nome do ambiente de configuração ativo (com fallback)**
- ☐ C) Define o nome do blueprint
- ☐ D) Obtém variável de ambiente do SO
- ☐ E) Importa o nome do pacote

Por que: `current_app` é o proxy para a app ativa. `.config.get(chave, default)` lê da config; se não existir, retorna o default.

3.

[Rotas]

Qual conversor de URL do Flask aceita barras "/" no valor capturado?

- ☐ A) int
- ☐ B) string
- ☐ C) float
- ☒ D) **path**
- ☐ E) uuid

Por que: `<path:x>` aceita barras. `string` (default) não. Útil para capturar caminhos como `/files/pasta/sub/arquivo.txt`.

4.

[venv]

Qual comando cria um ambiente virtual chamado `venv`?

- ☐ A) flask create venv
- ☐ B) python3 venv create
- ☒ C) **python3 -m venv venv**
- ☐ D) pip install venv
- ☐ E) python -m flask new-env

Por que: `python3 -m venv venv`. O `-m` executa o módulo `venv`, e o segundo `venv` é o nome da pasta que será criada.

5.

[Application Factory]

No padrão Application Factory, qual é o papel da função `create_app()`?

- ☐ A) Registrar as rotas diretamente em app.py
- ☒ B) **Criar e retornar uma instância configurada do Flask**
- ☐ C) Instalar o Flask e suas dependências
- ☐ D) Criar automaticamente um ambiente virtual
- ☐ E) Executar o servidor em produção

Por que: ``create_app()`` instancia o Flask sob demanda, aplica config, inicializa extensões, registra blueprints e retorna a app pronta.

6.

[Variáveis de ambiente]

Qual variável de ambiente indica o ponto de entrada da aplicação Flask?

- ☒ A) **FLASK_APP**
- ☐ B) FLASK_DEBUG
- ☐ C) FLASK_RUN
- ☐ D) FLASK_ENTRY
- ☐ E) APP_FLASK

Por que: ``FLASK_APP`` aponta o módulo/factory da aplicação. Aceita formatos como ``app``, ``app.py``, ou ``app:create_app("dev")``.

7.

[Blueprints]

O que são Blueprints no Flask?

- ☐ A) Um tipo especial de template HTML
- ☐ B) Sistema de cache do Flask
- ☐ C) Comando para instalar Flask
- ☐ D) Modo debug avançado
- ☒ E) **Mecanismo para separar funcionalidades em módulos reutilizáveis**

Por que: Blueprints permitem dividir a aplicação em módulos coesos (auth, admin, api etc.) que são registrados na app principal com ``app.register_blueprint()``.

8.

[Flask]

Qual biblioteca abaixo NÃO é dependência principal do Flask?

- ☐ A) Werkzeug
- ☐ B) Jinja2
- ☐ C) Click
- ☒ D) **SQLAlchemy**
- ☐ E) ItsDangerous

Por que: Flask puxa 5 dependências: Werkzeug, Jinja2, Click, ItsDangerous, Blinker. SQLAlchemy é extensão popular, mas não vem com o Flask.

9.

[Estrutura]

Em que arquivo deve ser definida a função ``create_app()``, segundo a estrutura modular padrão?

- ☒ A) **app/___init___py**
- ☐ B) config.py
- ☐ C) app.py
- ☐ D) main/routes.py
- ☐ E) venv/___init___py

Por que: ``app/__init__.py``. Este arquivo transforma a pasta ``app/`` em pacote e é o local convencional da `factory`.

10.

[Blueprints]

Qual das alternativas corresponde a uma rota registrada corretamente em um blueprint chamado ``main``?

- () A) `@app.route('/')`
- () B) `@route.main('/')`
- (X) C) `@main.route('/')`
- () D) `@flask.route('/')`
- () E) `@Blueprint.route('/')`

Por que: Rotas em blueprint usam ``@nome_do_bp.route(...)``. Aqui, ``@main.route('/')``.

11.

[venv]

Para ativar o venv no Linux/macOS:

- () A) `activate venv`
- () B) `venv/activate.sh`
- () C) `run venv`
- () D) `bash activate venv`
- (X) E) `source venv/bin/activate`

Por que: ``source venv/bin/activate``. No Windows: ``venv\Scripts\activate.bat`` (CMD) ou ``.\venv\Scripts\Activate.ps1`` (PowerShell).

Unidade 4 - Application Factory e Contexto

1.

[Application Factory]

Do ponto de vista arquitetural, qual é o principal objetivo do Application Factory no Flask?

- () A) Melhorar a performance do servidor HTTP
- () B) Evitar a criação de rotas dinâmicas
- (X) C) **Reduzir acoplamento e evitar importações circulares**
- () D) Substituir o uso de Blueprints
- () E) Eliminar a necessidade de configuração

Por que: A `factory` encapsula a criação da app numa função, quebrando o acoplamento global e evitando o loop de importação circular.

2.

[Contextos]

Sobre a relação entre Application Context e Request Context:

- ☐ A) Todo Application Context possui obrigatoriamente um Request Context
- ☒ B) **Todo Request Context possui um Application Context associado**
- ☐ C) Ambos são criados apenas na inicialização do servidor
- ☐ D) O Request Context existe antes do Application Context
- ☐ E) Eles são independentes

Por que: Regra de ouro: todo Request Context cria um Application Context. O inverso não: existe App Context sem Request (scripts, CLI, testes, cron).

3.

[CLI]

Considere: `def create_app(): app = Flask(__name__); return app`. Por convenção, o que `flask run` fará ao encontrar essa estrutura?

- ☒ A) **Executar automaticamente a função create_app**
- ☐ B) Criar um servidor sem contexto
- ☐ C) Ignorar a aplicação por não existir variável global app
- ☐ D) Exigir obrigatoriamente um arquivo run.py
- ☐ E) Compilar a aplicação antes de executar

Por que: O CLI do Flask reconhece automaticamente factories chamadas `create_app()` ou `make_app()`. Ele chama a função e usa o retorno.

4.

[Proxies]

O objeto `g` no Flask serve para:

- ☐ A) Configurar o servidor
- ☒ B) **Armazenamento temporário durante o ciclo da requisição (namespace em branco)**
- ☐ C) Armazenar dados persistentes globalmente para sempre
- ☐ D) Executar tarefas cron
- ☐ E) Substituir a sessão

Por que: `g` vive durante o Application Context. Usado para guardar coisas efêmeras: conexão de banco, usuário logado, dados calculados uma vez por requisição.

5.

[Exceções]

Que exceção é lançada ao acessar `current_app` sem contexto ativo?

- ☐ A) NameError
- ☒ B) **RuntimeError: Working outside of application context**
- ☐ C) ImportError
- ☐ D) TypeError
- ☐ E) ValueError

Por que: Sintoma clássico. Solução: envolver o código em `with app.app_context():`.

6.

[Arquitetura]

A Application Factory está associada a qual estilo arquitetural?

- ☐ A) Microserviços obrigatoriamente
- ☐ B) Serverless
- ☒ C) **Monólito Modular**
- ☐ D) Cliente-só
- ☐ E) P2P

Por que: Factory é padrão de monólito modular: um único processo, código organizado em módulos coesos (blueprints). Docker/Redis não mudam isso.

7.

[GoF]

A função `create_app()` é uma aplicação prática de qual padrão do GoF?

- ☐ A) Singleton
- ☐ B) Observer
- ☒ C) **Factory Method**
- ☐ D) Adapter
- ☐ E) Iterator

Por que: É Factory Method: encapsula a criação de objetos em um método, permitindo configurações variadas e desacoplamento.

8.

[Import circular]

Considere: `app.py` importa `init_app` de `views.py`, e `views.py` faz `from app import app`. Qual é o problema estrutural?

- ☐ A) Sobrecarga de memória
- ☐ B) Falha na renderização de templates
- ☐ C) Erro de tipagem dinâmica
- ☐ D) Incompatibilidade com WSGI
- ☒ E) **Dependência circular entre módulos**

Por que: É o cenário clássico de importação circular. Quando `views.py` tenta importar `app`, essa variável ainda não foi criada no `app.py`.

9.

[Lazy init]

Por que dizemos que a factory usa lazy initialization?

- ☒ A) **Porque a aplicação é criada apenas quando a função é invocada**
- ☐ B) Porque o Flask executa todas as rotas antecipadamente
- ☐ C) Porque as extensões são removidas da aplicação
- ☐ D) Porque o Python adia a compilação do código
- ☐ E) Porque o servidor executa em modo assíncrono

Por que: Ao importar o módulo, só a definição da função é carregada. Nenhum objeto Flask é instanciado até `create_app()` ser chamada.

10.

[Boas práticas]

Em projetos Flask com factory, qual é considerada boa prática arquitetural?

- () A) Importar o objeto app em todos os módulos
- () B) Declarar o app como global obrigatória
- () C) Criar múltiplos objetos Flask globais
- (X) D) Evitar importar diretamente o objeto app nos módulos**
- () E) Substituir o app por variáveis de ambiente

Por que: Módulos não devem importar diretamente o `app` global. Recebem-no via injeção (parâmetro de função como `init\_app(app)`) ou usam o proxy `current\_app`.

11.

[init_app]

Qual a principal função do padrão `init\_app(app)`?

- () A) Criar múltiplas instâncias do Flask
- (X) B) Registrar componentes na aplicação de forma desacoplada**
- () C) Executar requisições HTTP automaticamente
- () D) Substituir o uso de decorators
- () E) Configurar o servidor WSGI

Por que: `init\_app(app)` é o padrão para extensões e módulos receberem a app como parâmetro (injeção de dependência), em vez de importá-la.

12.

[App Context]

O código `valor = current\_app.config["DEBUG"]` só funciona corretamente quando:

- () A) O servidor está em modo debug
- () B) O banco de dados está conectado
- () C) O Flask está rodando com Gunicorn
- (X) D) Existe um contexto de aplicação ativo**
- () E) O arquivo config.py foi importado

Por que: `current\_app` é um proxy que só resolve se houver Application Context ativo. Sem ele: `RuntimeError: Working outside of application context`.

13.

[Request Context]

Qual descreve corretamente o Request Context?

- () A) Estado global permanente da aplicação
- () B) Contexto criado apenas durante compilação
- (X) C) Contexto criado a cada requisição HTTP**
- () D) Contexto exclusivo do banco de dados
- () E) Substituto do Application Factory

Por que: Request Context é ativado a cada requisição HTTP, empilhado sobre o Application Context, e expõe `request` e `session`.

14.

[Proxies]

Objetos como ``request``, ``current_app`` e ``g`` são classificados como:

- ☐ A) Variáveis globais estáticas
- ☐ B) Middlewares HTTP
- ☐ C) Decoradores de rota
- ☐ D) Objetos WSGI diretos
- ☒ E) **Proxies de contexto (context-local proxies)**

Por que: São proxies context-local baseados em Werkzeug. Cada thread/requisição enxerga o "seu" objeto, mesmo usando o nome global.

15.

[Setup]

Durante a fase de configuração (setup) da aplicação Flask, qual ação é adequada?

- ☐ A) Acessar dados do objeto request
- ☐ B) Processar formulários HTTP
- ☒ C) **Registrar blueprints e extensões**
- ☐ D) Manipular sessões de usuário
- ☐ E) Executar queries de requisição ativa

Por que: Setup é o momento antes das requisições. Aqui você configura `app.config`, registra blueprints, inicializa extensões (`init_app`), define hooks.